Lesson 9 -Tools, Knowledge and Memory

Part 1 –Tools and Knowledge

Agenda

- Tools
  - Characteristics of Tools
  - Built-in-tools
  - Custom Tools
    - Using @tool
    - Subclassing BaseTool
- Hierarchical process
- Sequential vs hierarchical
- Knowledge to Agents and Crew
  - Text Sources(Raw string, .txt, .pdf)
  - Structured Data(csv, excel, json)
  - Web Content(CrewDocling, WebSiteSearch)
- Memory
- Human in the loop

Tools in CrewAI

In CrewAI, a **tool** is a capability or function that agents use to perform tasks from simple searches to complex interactions—while supporting smooth teamwork.

Key Characteristics of Tools

- **Utility:** Crafted for tasks such as web searching, data analysis, content generation, and agent collaboration.
- **Integration:** Boosts agent capabilities by seamlessly integrating tools into their workflow.
- **Customizability:** Provides the flexibility to develop custom tools or utilize existing ones, catering to the specific needs of agents.
- **Error Handling:** Incorporates robust error handling mechanisms to ensure smooth operation.
- **Caching Mechanism:** Features intelligent caching to optimize performance and reduce redundant operations. Enabling agents to efficiently reuse previously obtained results.
- **Asynchronous Support:** CrewAI supports asynchronous tools, allowing you to implement tools that perform non-blocking operations like network requests, file I/O, or other async operations without blocking the main execution thread.

Built-in Tools

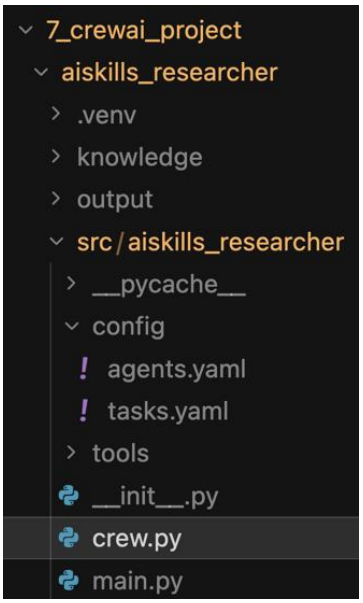
CrewAI provides built-in tools through the `crewai[tools]` package.

Documentation:
https://docs.crewai.com/en/concepts/tools#available-crewai-tools

Install:

pip install 'crewai[tools]'. Or uv add "crewai[tools]"

Refer Demo: Under the folder 7_crewai_project(Tools & Context)/ aiskills_researcher



About this code:

This code defines a **CrewAI sequential workflow** to produce an **AI skills report for software developers (2026)** using three agents:

- **Researcher agent** uses **SerperDevTool** to search the web and generate research notes with sources.
- **Analyst agent** turns the research into structured categories and key insights.
- **Report writer agent** creates a user-friendly final report.

The SerperDevTool performs internet searches using the serper.dev API. Here's how to use it:

Setup:

1. Install: **pip install 'crewai[tools]'. Or uv add "crewai[tools]"**
2. Get a free API key at serper.dev
3. Set the SERPER_API_KEY environment variable

Usage:
from crewai_tools import SerperDevTool
tool = SerperDevTool()

The tasks run **in order (research → analysis → report)**. **Tasks can reuse earlier outputs via context** (e.g., the **analysis_task** can be configured to take the **research_task output** as input context), so the report_writer writes based on what the researcher already found instead of re-searching.

Finally, run() kicks off the crew with role and year, prints the final result, and saves it to output/ai_skills_2026.md.

Custom Tools (Creating Your Own Tools)

There are **two main ways** to create a tool in CrewAI:

1) Using the `@tool` Decorator

When to use: Quick and simple tools with minimal setup.

Key Points

- Simplest approach: decorate a function
- Function docstring becomes the tool description
- Tool name is passed as a decorator argument

Template

```
from crewai.tools import tool
@tool("Tool Name")
def my_tool(argument: str) -> str:
    """Description of what this tool does - agents use this to decide
when to use it."""
    # Your logic here
    return "result"
```

Example

```
import requests
from crewai.tools import tool
@tool("Get Weather")
def get_weather(city: str) -> str:
    """Fetch current weather for a given city from wttr.in."""
    endpoint = "https://wttr.in"
    response = requests.get(f"{endpoint}/{city}")
    return response.text
```

2) Subclassing `BaseTool`

When to use: Complex tools that need input validation, custom schemas, or more control.

Key Points

- Define an input schema using **Pydantic** (validation)
- More explicit control over tool name, description, and arguments
- Best for tools with multiple/complex parameters

Template

```
from typing import Type
from crewai.tools import BaseTool
from pydantic import BaseModel, Field

class MyToolInput(BaseModel):
    """Input schema for the tool."""
    argument: str = Field(..., description="Description of argument")

class MyCustomTool(BaseTool):
    name: str = "Tool Name"
    description: str = "What this tool does"
    args_schema: Type[BaseModel] = MyToolInput

    def _run(self, argument: str) -> str:
        # Your logic here
        return "result"
```

Here's how execution works when you assign a tool to an agent:

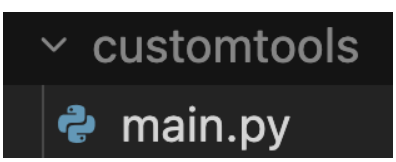
1. Tool Assignment – You pass the tool in the agent's tools list: tools=[MyCustomTool()]
2. Task Execution – When the agent receives a task, the LLM decides if and when to call the tool based on the task description and the tool's name/description.
3. Tool Invocation – The LLM provides arguments matching your args_schema, and CrewAI calls _run() with those arguments.
4. Result Handling – The string output is returned to the agent, which then reflects on it and may use it to formulate its final answer.

Example

```
from typing import Type
from crewai.tools import BaseTool
from pydantic import BaseModel, Field
import requests

class WeatherInput(BaseModel):
    """Input schema for WeatherTool."""
    city: str = Field(..., description="City name to get weather for")

class WeatherTool(BaseTool):
    name: str = "Get Weather"
    description: str = "Fetch current weather for a given city from
wttr.in."
    args_schema: Type[BaseModel] = WeatherInput

    def _run(self, city: str) -> str:
        endpoint = "https://wttr.in"
        response = requests.get(f"{endpoint}/{city}")
        return response.text
```

Refer Demo: Under the folder 7_crewai_project

This demonstration was implemented without utilizing a configuration file (.yaml). The agents and tasks are explicitly defined in the code, without the use of decorators. For smaller tasks or projects, this direct approach can be efficient and practical.



About the Demo Code(Custom Tools)

CrewAI workflow with two agents and two tasks, then runs them **sequentially**:

- **Tool 1 (decorator tool):** `get_weather(city)` calls `wttr.in` and returns a short weather string for a city.
- **Tool 2 (custom BaseTool):** `StockPriceTool` accepts a ticker symbol(AAPL - Apple) and returns a **mock** price string (always `$150.25` in this example, you can use Api call to retrieve live stock).
- **Weather Agent** uses the `get_weather` tool to handle the task **"Get weather for New York."**

- **Stock Agent** uses `StockPriceTool()` to handle the task **"Get AAPL stock price."**
- A **Crew** is created with both agents and tasks; when `crew.kickoff()` runs, it executes the weather task first, then the stock task, prints verbose logs, and finally prints the combined result.

Visual Flow of Tool Execution

```
Agent receives task
    ↓
Agent analyzes task → Determines tool is needed
    ↓
Agent selects tool (based on name & description)
    ↓
Agent prepares tool input parameters
    ↓
Input validated against args_schema (Pydantic)
    ↓
Tool._run() method executes
    ↓
Tool returns result
    ↓
Agent interprets result
    ↓
Agent decides:
    ├→ Use result to complete task → Task finished
    └→ Need more information → Call another tool or continue reasoning
```

Hierarchical Process

In a **hierarchical process**, you typically do **not** create a manager agent yourself (unless you want a custom one). CrewAI automatically creates a manager that:

1. **Coordinates the workflow** — decides which agent handles each task
2. **Delegates tasks** — assigns work based on agent roles and capabilities

3. **Validates outcomes** — reviews results before moving forward

Default manager when you want a fast and simple hierarchical setup
custom manager agent when you need stronger control, clarity, and workflow customization

Example – manager_llm for Hierarchical Process

```
crew = Crew(
    agents=[weather_agent, stock_agent],
    tasks=[weather_task, stock_task, report_task],
    process=Process.hierarchical,
    manager_llm="gpt-4o",  # Manager is auto-created using this LLM
    verbose=True
)
```

What Happens

- The manager LLM automatically acts as the coordinator for the crew.
- It decides which agent should handle each task and manages the task flow.
- It collects and organizes the agent outputs to help produce the final result.

Example – Custom Manager for Hierarchical Process

```
manager = Agent(
    role="Student Support Manager",
    goal="Understand the student query and delegate it to the most suitable specialist agent.",
    backstory=(
        "You are an academic support coordinator. "
        "Your job is to review the student's request, decide which specialist is the best fit, "
        "and ensure the final response is relevant and focused."
    ),
    verbose=True,
    allow_delegation=True
)
```

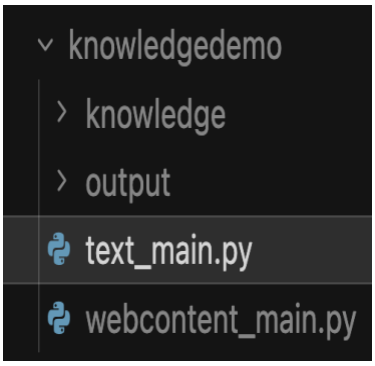
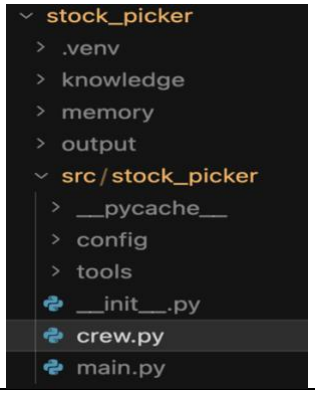
```
student_support_crew = Crew(
    agents=[tutor_agent, study_planner_agent, quiz_creator_agent],
    tasks=[explain_task, study_plan_task, quiz_task],
    process=Process.hierarchical,
    manager_agent=manager,
    planning=True,
```

```
    verbose=True
)
```

What Happens

- The manager agent reads the student query and understands what kind of help is needed.
- It delegates the work to the most suitable specialist agent such as tutor, study planner, or quiz creator.
- It coordinates the overall flow, so the final response is relevant, focused, and well organized

Refer Demo: 7_crewai_project

manager_llm hierarchicalprocess > main1.py	**Custom Manager LLM** hierarchicalprocess > main2.py
mangaer_llm & knowledge using hierarchical workflow. Refer: text_main.py 	**manager_agent, knowledge path customization and hierarchical workflow** 

Sequential vs Hierarchical

Feature	Sequential	Hierarchical
Task order	Fixed order	Manager decides task allocation
Assignment	Pre-assigned to agents	Dynamic delegation based on capabilities
Nature	Simple, predictable	Flexible, adaptive

Knowledge in CrewAI

Knowledge in CrewAI allows agents to access and use external information sources during tasks. Think of it as giving agents a **reference library** they can consult while working.

Key Benefits of Knowledge

- Enhance agents with domain-specific information
- Support decisions with real-world data
- Maintain context across conversations due to knowledge persist on the ChromaDB.
- Ground responses in factual information

Knowledge Sources Supported by CrewAI

Text Sources

- Raw strings
- Text files (`.txt`)
- PDF documents

Structured Data

- CSV files
- Excel spreadsheets
- JSON documents

Web Content Knowledge Options

- **CrewDoclingSource (as in your code):** Uses URLs directly as knowledge sources (web pages/documents).
- **WebsiteSearchTool:** Performs semantic RAG searches within website content during execution.
- **Custom Knowledge Sources:** Extend `BaseKnowledgeSource` to fetch and process web content.

Behind the Scenes: How Knowledge Retrieval Works

Regardless of the source type — Text, Structured Data, or Web Content — CrewAI processes all knowledge through the same RAG (Retrieval-Augmented Generation) pipeline:

1. Ingest — Each source type has its own parser (e.g., PDF parser, CSV parser, web scraper) that extracts raw text from the format.
2. Chunk — Extracted text is split into smaller, manageable pieces.

3. Embed — Each chunk is converted into a numerical vector using an embedding model.
4. Store — Vectors are stored in ChromaDB, CrewAI's default embedded vector database (bundled automatically, zero configuration — similar to how SQLite is a default embedded database in OpenAI Agents frameworks).
5. Retrieve — At runtime, the agent's query is semantically matched against stored vectors, retrieving only the most relevant chunks.
6. Augment — Retrieved chunks are injected into the agent's prompt for accurate, context-aware responses.
7. Generate —The retrieved information is then used by the agent's LLM to produce its response.

Key Points

- This is not regular file reading — CrewAI never passes entire files to the LLM.
- Only the ingestion step differs across source types; the rest of the pipeline is identical.
- ChromaDB is included with CrewAI — no separate installation required.
- Two approaches available:
    - Implicit → Use knowledge_sources on Crew/Agent (ChromaDB managed automatically)
    - Explicit → Use get_rag_client() for direct vector store control. (Not covered)

Built-in Parsers (No Extra Install)

Each source type has its own dedicated class under crewai.knowledge.source:

Source Type	Parser Class	Import Path
Raw strings	StringKnowledgeSource	crewai.knowledge.source.string_knowledge_source
Text files	TextFileKnowledgeSource	crewai.knowledge.source.text_file_knowledge_source
PDF	PDFKnowledgeSource	crewai.knowledge.source.pdf_knowledge_source
CSV	CSVKnowledgeSource	crewai.knowledge.source.csv_knowledge_source
Excel	ExcelKnowledgeSource	crewai.knowledge.source.excel_knowledge_source
JSON	JSONKnowledgeSource	crewai.knowledge.source.json_knowledge_source

External Library — Docling (Requires uv pip install docling)

Source Type	Parser Class	Import Path
Web content / URLs	CrewDoclingSource	crewai.knowledge.source.crew_docling_source

- Docling is the external library CrewAI uses for web content parsing.
- **It requires separate installation**: **uv pip install docling**

Crew-Level vs Agent-Level Knowledge

Key Differences

Crew-Level Knowledge	Agent-Level Knowledge
Shared by all agents	Used only by a specific agent
Set via `Crew(knowledge_sources=[...])`	Set via `Agent(knowledge_sources=[...])`
Good for policies/general info	Good for specialized role-specific info

Step-by-Step: Using Text Sources in CrewAI

Step 1: Set Up Project Structure

If you create a project using `crewai create crew`, the `knowledge` folder is created automatically. Otherwise, create a **`knowledge/`** folder in your project root.

Step 2: Import the Required Knowledge Source

```
# For raw strings
from crewai.knowledge.source.string_knowledge_source import
StringKnowledgeSource

# For text files (.txt)
from crewai.knowledge.source.text_file_knowledge_source import
TextFileKnowledgeSource

# For PDF documents
from crewai.knowledge.source.pdf_knowledge_source import
PDFKnowledgeSource
```

Step 3: Create the Knowledge Source Object

Raw String

```
string_source = StringKnowledgeSource(
    content="Your raw text content goes here."
)
```

Text Files (.txt)

Place files in `./knowledge` and use only the filename.

```
text_source = TextFileKnowledgeSource(
    file_paths=["document.txt", "notes.txt"]
)
```

PDF Documents

Place files in `./knowledge` and use only the filename.

```
pdf_source = PDFKnowledgeSource(
    file_paths=["report.pdf", "manual.pdf"]
)
```

Step 4: Add Knowledge Source to Agent or Crew

Agent-Level (specific to one agent)

```
agent = Agent(
    role="Document Analyst",
    goal="Analyze documents and answer questions",
    backstory="Expert in document analysis",
    knowledge_sources=[text_source, pdf_source]
)
```

Crew-Level (shared by all agents)

```
crew = Crew(
    agents=[agent1, agent2],
    tasks=[task1, task2],
    knowledge_sources=[string_source]
)
```

Step 5 (Optional): Configure Knowledge Settings

```
from crewai.knowledge.knowledge_config import KnowledgeConfig

knowledge_config = KnowledgeConfig(
    results_limit=10,        # default: 3
    score_threshold=0.5      # default: 0.35
)
```

```
agent = Agent(
    role="Analyst",
    goal="Analyze data",
    backstory="Expert analyst",
    knowledge_sources=[text_source],
    knowledge_config=knowledge_config )
```

- **results_limit** controls how many chunks the agent sees. Setting it to 10 means the agent receives 10 most relevant chunks instead of 3.
- **score_threshold** controls how relevant those chunks must be. Setting it to 0.5 means only chunks with 50%+ similarity to the query are returned.

Quick Reference Summary

Step	Action
1	Create `./knowledge` folder in project root
2	Import the correct knowledge source class
3	Create source object (use filename only for files)
4	Add to `knowledge_sources=[]` in Agent or Crew
5	(Optional) Configure using `KnowledgeConfig`

Structured Data Knowledge Sources

1) CSV Files

```
from crewai.knowledge.source.csv_knowledge_source import
CSVKnowledgeSource

csv_source = CSVKnowledgeSource(
    file_paths=["data.csv", "records.csv"]
)
```

2) Excel Spreadsheets

Excel source supports multi-tab files.

```
from crewai.knowledge.source.excel_knowledge_source import
ExcelKnowledgeSource

excel_source = ExcelKnowledgeSource(
    file_paths=["spreadsheet.xlsx", "report.xlsx"]
```

```
)
```

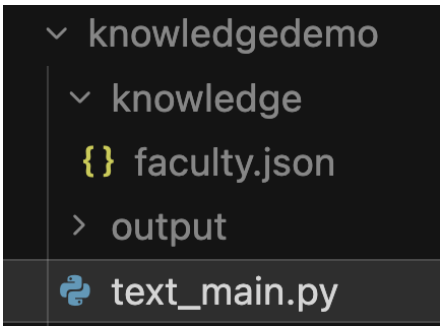
3) JSON Documents

```
from crewai.knowledge.source.json_knowledge_source import
JSONKnowledgeSource

json_source = JSONKnowledgeSource(
    file_paths=["data.json", "config.json"]
)
```

Place your files under the `knowledge` folder.

Refer Demo: 7_crewai_project

.txt and .json knowledge



About Demo:

This code builds a **hierarchical CrewAI Q&A crew** with two agents: one answers from **department text knowledge**, and the other answers from **faculty JSON data**. A **manager LLM** decides which agent to use for each question, and it runs two queries (one about who teaches a subject, one about the Data Science track) and prints the results.

Web Contents

CrewDoclingSource

A knowledge source that uses the **Docling** library to extract content from web URLs and make it available as **crew-wide knowledge**. It pre-loads web content into the crew's knowledge base.

Purpose

- Pre-load web content into a knowledge base for RAG queries.

How It Works

1. Fetches and processes web pages during initialization
2. Stores content in a vector database (embeddings)
3. Agents query the pre-loaded knowledge during execution

Best For

- Static content that does not change frequently
- When you know the exact URLs upfront
- Faster queries because content is already indexed

```
from crewai.knowledge.source.crew_docling_source import
CrewDoclingSource

content_source = CrewDoclingSource(
    file_paths=[
        "https://lilianweng.github.io/posts/2024-11-28-reward-
hacking",
    ],
)

crew = Crew(
    agents=[agent],
    tasks=[task],
    knowledge_sources=[content_source],
)
```

WebsiteSearchTool

A RAG tool agents use **during execution** for semantic searches inside website content (on-demand searching).

```
from crewai_tools import WebsiteSearchTool

tool = WebsiteSearchTool(website='https://example.com')

agent = Agent(
    role="Researcher",
    tools=[tool],
)
```

Key Difference

- **CrewDoclingSource:** Loads and indexes content **before** execution
- **WebsiteSearchTool:** Searches content **during** agent execution

CrewDoclingSource vs WebsiteSearchTool

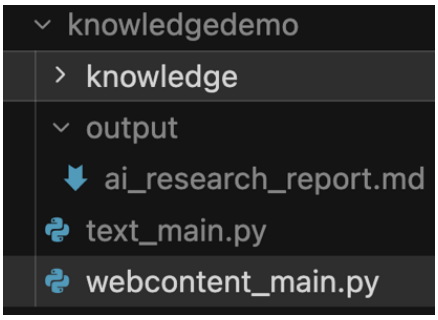
Aspect	CrewDoclingSource	WebsiteSearchTool
Type	Knowledge Source	RAG Tool
When content is loaded	Before crew execution (pre-loaded)	During agent execution (on-demand)
Scope	Crew-wide	Agent-specific
Use case	Static reference content all agents need	Dynamic searching during execution
Configuration	`Crew(knowledge_sources=[...])`	`Agent(tools=[...])`
Dependency	Requires docling package	Included in `crewai[tools]`
Content discovery	URLs must be specified upfront	Can search websites at runtime

---

When to Use Each

Scenario	Use
Agents need consistent access to specific documents	CrewDoclingSource
Agent needs to search websites dynamically	WebsiteSearchTool
Content is known beforehand	CrewDoclingSource
Content URLs determined during execution	WebsiteSearchTool
Building a crew-wide knowledge base	CrewDoclingSource
Performing ad-hoc web research	WebsiteSearchTool

Refer Demo: 7_crewai_project for CrewDoclingSource(webcontent_main.py)



About webcontent_main.py project

This code builds a **2-step CrewAI pipeline** that reads two AI blog/paper-style webpages into a **local knowledge store**, then uses two agents to produce a report:

- **Knowledge loading:** `CrewDoclingSource` ingests the two provided URLs (reward hacking + hallucination) and stores them in the CrewAI knowledge DB (path printed).
- **Agent 1 (Researcher):** analyzes the loaded content for the given `{topic}` and extracts **key concepts, methods, findings, and limitations**.
- **Agent 2 (Writer):** uses the researcher's output as **context** to write a polished **executive summary report + recommendations**, saving it to `output/ai_research_report.md`.
- **Execution:** runs tasks **sequentially** via `crew.kickoff(...)` and prints the final raw result.

CrewAI Memory

In CrewAI, setting memory=True at the crew level enables **shared memory** for the whole crew. CrewAI automatically creates a default Memory() object, stores useful facts after tasks, and recalls relevant context before later tasks. This helps agents remember user preferences, prior decisions, important facts, and past interaction context.

By default, all agents share the same crew memory unless a specific agent is given its own memory. Agent-level memory is useful when you want **private or scoped memory** for one agent.

In simple terms, memory makes agents more **personalized, consistent, and less repetitive**.

What Happens Internally

When memory=True is enabled:

- CrewAI creates a default Memory() instance.
- All agents share that memory.
- Data is stored locally using embedded vector database **LanceDB**, usually at ./.crewai/memory, so memory can persist across runs.

For each task:

- **Before execution:** relevant memory is recalled and added to the prompt.
- **After execution:** useful facts are extracted from the output and saved.

During saving, CrewAI uses embeddings for semantic search, checks for duplicates, and stores memories in the background. During recall, memories are ranked using **semantic similarity, recency, and importance**. At shutdown, pending saves are completed to avoid data loss.

Ref: Stock Picker Demo

Custom Memory

Instead of just using memory=True, you can create a custom Memory() object and pass it to the crew for more control over recall and storage behavior.

```
from crewai import Crew, Memory

memory = Memory(
    semantic_weight=0.4,
    recency_weight=0.4,
    importance_weight=0.2,
    recency_half_life_days=14,
)

crew = Crew(
    agents=[...],
    tasks=[...],
    memory=memory,
)
```

Important Parameters

The main Memory() parameters are:

- semantic_weight – gives priority to meaning similarity
- recency_weight – gives priority to newer memories
- importance_weight – gives priority to more important facts
- recency_half_life_days – controls how fast old memories lose value
- consolidation_threshold – checks similar memories for duplication
- confidence_threshold_high / low – controls shallow vs deeper recall
- exploration_budget – sets extra exploration rounds during deep recall

Shallow recall: Fast vector search with no LLM calls.
Deep recall: Multi-step intelligent retrieval with LLM analysis, confidence routing, and optional exploration rounds.

Summary
CrewAI memory allows agents to **store, recall, and reuse useful information** across tasks and runs. Shared crew memory supports collaboration, while agent-level memory supports privacy or specialization. Default memory works automatically, and custom memory gives more control when needed.

Ref: https://docs.crewai.com/en/concepts/memory

Human-In-The-Loop (HITL)

Human-In-The-Loop (HITL) is a powerful approach that combines artificial intelligence with human expertise to enhance decision-making and improve task outcomes.

Best Practices

- Be Specific: Provide clear, actionable feedback that directly addresses the task at hand
- Stay Relevant: Only include information that will help improve the task execution
- Be Timely: Respond to HITL prompts promptly to avoid workflow delays
- Review Carefully: Double-check your feedback before submitting to ensure accuracy

Common Use Cases

HITL workflows are particularly valuable for:

- Quality assurance and validation
- Complex decision-making scenarios
- Sensitive or high-stakes operations
- Creative tasks requiring human judgment
- Compliance and regulatory reviews

HITL Implementation

Just set human_input=True on any task where you want human review.

What happens at runtime:

1. The agent works on the task normally.
2. Before delivering its final answer, it pauses and prompts you in the terminal for input.
3. You can provide extra context, clarify ambiguities, or validate the output.
4. The agent incorporates your feedback and produces the final result.

Refer the Demo:

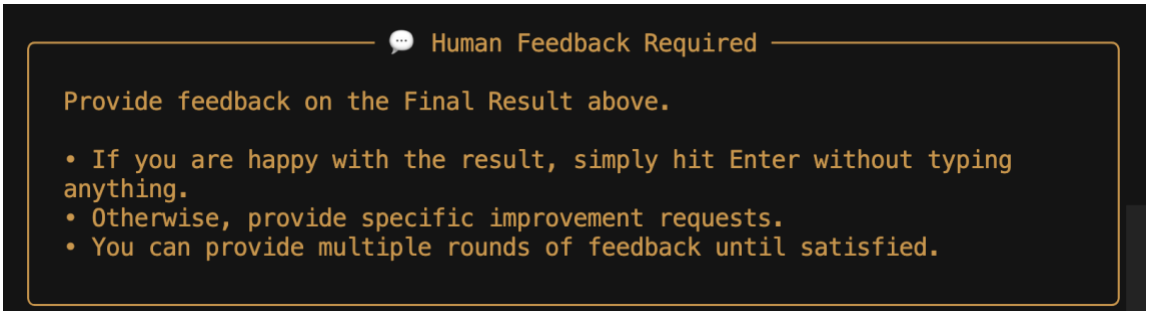
What happens step by step:

Step	Agent	What happens	Human involved?
1	Researcher	Gathers AI trends	☑ You review the research
2	Editor	Fact-checks & cleans	✕ Automatic
3	Writer	Writes the blog post	☑ You approve the final draft

Task 1 — You catch bad research *early* before it flows downstream.
Task 2 — Not every step needs human review; let agents handle routine work.
Task 3 — You do a final quality check before the blog is saved to blog_post.md.

This is the core pattern: put human_input=True only on tasks where human judgment adds the most value.

Screenshots from Console for HITL approval



```
💬 Human Feedback Required

Provide feedback on the Final Result above.

• If you are happy with the result, simply hit Enter without typing
anything.
• Otherwise, provide specific improvement requests.
• You can provide multiple rounds of feedback until satisfied.
```